

Fresher Android

Classes and objects





Classes |

- A class in Kotlin can have a *primary constructor*:

```
class Person constructor(firstName: String) { /*...*/ }
```

```
class Person(firstName: String) { /*...*/ } // No annotations and visibility modifiers
```

- Initialization code can be placed in *initializer blocks*:

```
init {  
    /*...*/  
}
```

- Parameters of the primary constructor can be used:

- ☐ in the initializer blocks
- ☐ in property initializers declared in the class body

- Concise syntax for declaring properties and initializing them from the primary constructor

```
class Person(val firstName: String, val lastName: String, var isEmployed: Boolean = true)
```

- A class in Kotlin can have one or more *secondary constructors*:

```
class Pet {  
    constructor(owner: Person) {  
        owner.pets.add(this) // adds this pet to the list of its owner's pets  
    }  
}
```

- If the class has a primary constructor, each secondary constructor needs to delegate to the primary constructor:

```
class Person(firstName: String) {  
    constructor(firstName:String, lastName: String) : this (firstName) {  
        /* ... */  
    }  
}
```

- If a non-abstract class does not declare any constructors (primary or secondary), it will have a generated primary constructor with no arguments. The visibility of the constructor will be public.
- If you don't want your class to have a public constructor, declare an empty primary constructor with non-default visibility:

```
class DontCreateMe private constructor () { /* ... */ }
```

Creating instances of classes

Kotlin does not have a **new** keyword.

- **val** invoice = Invoice()
- **val** customer = Customer(**"Joe Smith"**)

- Classes can contain:
 - ❑ Constructors and initializer blocks
 - ❑ Functions
 - ❑ Properties
 - ❑ Nested and inner classes
 - ❑ Object declarations

- A class and some of its members may be declared **abstract**
- An abstract member does not have an implementation in its class
- Don't need to annotate an abstract class or function with **open**
- can override a non-abstract **open** member with an abstract one

```
open class Polygon {  
    open fun draw() {}  
}
```

```
abstract class Rectangle : Polygon() {  
    abstract override fun draw()  
}
```




| Inheritance

- All classes in Kotlin have a common superclass **Any**

class Example // Implicitly inherits from Any

- **Any** has three methods:

- ❑ **equals()**

- ❑ **hashCode()**

- ❑ **toString()**

- By default, Kotlin classes are final: they can't be inherited

⇒ To make a class inheritable, mark it with the **open** keyword.

open class Base // Class is open for inheritance

- To declare an explicit supertype, place the type after a colon in the class header:

open class Base(p: Int)

class Derived(p: Int) : Base(p)

- Using **override** modifier for overridable members and overrides

```
open class Shape {  
    open fun draw() { /*...*/ }  
    fun fill() { /*...*/ }  
}  
  
class Circle() : Shape() {  
    override fun draw() { /*...*/ }  
}
```

Overriding properties

- Overriding properties works in a similar way to overriding methods

```
open class Shape {  
    open val vertexCount: Int = 0  
}
```

```
class Rectangle : Shape() {  
    override val vertexCount = 4  
}
```

- Can also override a **val** property with a **var** property

```
interface Shape {  
    val vertexCount: Int  
}
```

```
class Polygon : Shape {  
    override var vertexCount: Int = 0 // Can be set to any number later  
}
```

Derived class initialization order

- Base class constructor
- Base class's properties initialization
- Derived class constructor
- Derived class's properties initialization

Calling the superclass implementation

- Code in a derived class can call its superclass functions and property accessors implementations using the **super** keyword

```
open class Rectangle {  
    open fun draw() { println("Drawing a rectangle") }  
    val borderColor: String get() = "black"  
}
```

```
class FilledRectangle : Rectangle() {  
    override fun draw() {  
        super.draw()  
        println("Filling the rectangle")  
    }  
  
    val fillColor: String get() = super.borderColor  
}
```


Overriding rules

- If a class inherits multiple implementations of the same member from its immediate superclasses, it must override this member and provide its own implementation

```
open class Rectangle {  
    open fun draw() { /* ... */ }  
}
```

```
interface Polygon {  
    fun draw() { /* ... */ } // interface members are 'open' by default  
}
```

```
class Square() : Rectangle(), Polygon {  
    // The compiler requires draw() to be overridden:  
    override fun draw() {  
        super<Rectangle>.draw() // call to Rectangle.draw()  
        super<Polygon>.draw() // call to Polygon.draw()  
    }
```



Properties |

Declaring properties

- Properties in Kotlin classes can be declared either as mutable using the **var** keyword, or as read-only using the **val** keyword.

```
class Address {  
    var name: String = "Holmes, Sherlock"  
    var street: String = "Baker"  
    var city: String = "London"  
    var state: String? = null  
    var zip: String = "123456"  
}
```

- To use a property, simply refer to it by its name:

```
fun copyAddress(address: Address): Address {  
    val result = Address() // there's no 'new' keyword in Kotlin  
    result.name = address.name // accessors are called  
    result.street = address.street  
    // ...  
    return result  
}
```

Getters and setters

- The full syntax for declaring a property:

```
var <propertyName>[: <PropertyType>] [= <property_initializer>]  
    [<getter>]  
    [<setter>]
```

- For example:

```
var stringRepresentation: String  
    get() = this.toString()  
    set(value) {  
        setDataFromString(value)  
    }
```

- **val** does not allow a setter

```
val isEmpty: Boolean  
    get() = this.size == 0
```

- Change the visibility of an accessor or to annotate it

```
var setterVisibility: String = "abc"  
    private set // the setter is private and has the default implementation  
var setterWithAnnotation: Any? = null
```

- **@Inject set** // annotate the setter with Inject

Backing field

- A field is only used as a part of a property to hold its value in memory

```
var counter = 0
  set(value) {
    if (value >= 0)
      field = value
  }
  get() {
    return field
  }
```

- If you want to do something that does not fit into this implicit backing field scheme, you can always fall back to having a backing property:

```
private var _table: Map<String, Int>? = null
public val table: Map<String, Int>
    get() {
        if (_table == null) {
            _table = HashMap() // Type parameters are inferred
        }
        return _table ?: throw AssertionError("Set to null by another thread")
    }
```

- If the value of a read-only property is known at the compile time, mark it as a compile time constant using the **const** modifier
 - ❑ Top-level, or member of an object declaration or a companion object.
 - ❑ Initialized with a value of type `String` or a primitive type
 - ❑ No custom getter

```
const val SUBSYSTEM_DEPRECATED: String = "This subsystem is deprecated"  
@Deprecated(SUBSYSTEM_DEPRECATED) fun foo() { ... }
```


- Mark the property with the lateinit modifier

```
public class MyTest {  
    lateinit var subject: TestSubject  
}
```
- The modifier can be used on **var** properties:
 - ☐ declared inside the body of a class
 - ☐ not in the primary constructor
 - ☐ only when the property does not have a custom getter or setter
 - ☐ for top-level properties
 - ☐ local variables
- The type of the property or variable must be non-null, and it must not be a primitive type.
- Accessing a **lateinit** property before it has been initialized throws a special exception

Checking whether a lateinit var is initialized

- use **.isInitialized** on the reference to that property:

```
if (foo::bar.isInitialized) {  
    println(foo.bar)  
}
```



| Interfaces



- Contain:
 - ❑ declarations of abstract methods
 - ❑ method implementations
- Different from abstract classes is that interfaces cannot store state
- Can have properties but these need to be abstract or to provide accessor implementations.
- An interface is defined using the keyword **interface**

```
interface MyInterface {  
    fun bar()  
    fun foo() {  
        // optional body  
    }  
}
```

- A class or object can implement one or more interfaces

```
class Child : MyInterface {  
    override fun bar() {  
        // body  
    }  
}
```

Properties in interfaces

- A property declared in an interface can either be abstract, or it can provide implementations for accessors
- Properties declared in interfaces can't have backing fields

```
interface MyInterface {  
    val prop: Int // abstract
```

```
    val propertyWithImplementation: String  
    get() = "foo"
```

```
    fun foo() {  
        print(prop)  
    }  
}
```

```
class Child : MyInterface {  
    override val prop: Int = 29  
}
```

- Classes implementing such an interface are only required to define the missing implementations:

```
interface Named {  
    val name: String  
}
```

```
interface Person : Named {  
    val firstName: String  
    val lastName: String
```

```
    override val name: String get() = "$firstName $lastName"  
}
```

```
data class Employee(  
    // implementing 'name' is not required  
    override val firstName: String,  
    override val lastName: String,  
    val position: Position  
): Person
```


Resolving overriding conflicts

```
interface A {  
    fun foo() { print("A") }  
    fun bar()  
}  
  
interface B {  
    fun foo() { print("B") }  
    fun bar() { print("bar") }  
}  
  
class C : A {  
    override fun bar() { print("bar") }  
}  
  
class D : A, B {  
    override fun foo() {  
        super<A>.foo()  
        super<B>.foo()  
    }  
  
    override fun bar() {  
        super<B>.bar()  
    }  
}
```



Visibility modifiers |

Visibility modifiers

- There are four visibility modifiers in Kotlin:
 - ❑ **private**: visible inside this class only (including all its members)
 - ❑ **protected**: same as private but is also visible in subclasses
 - ❑ **internal**: visible everywhere in the same module
 - ❑ **public**: visible everywhere
- The default visibility is **public**



| Extensions

- To declare an extension function, prefix its name with a receiver type, that means the type being extended.
`fun` MutableList<Int>.swap(index1: Int, index2: Int) {
 `val` tmp = `this`[index1] // 'this' corresponds to the list
 `this`[index1] = `this`[index2]
 `this`[index2] = tmp
}
- An extension function being called is determined by the type of the expression on which the function is invoked, not by the type of the result of evaluating that expression at runtime

`open class` Shape

`class` Rectangle: Shape()

`fun` Shape.getName() = "Shape"

`fun` Rectangle.getName() = "Rectangle"

```
fun printClassName(s: Shape) {  
    println(s.getName())  
}
```

printClassName(Rectangle())

- Extensions can be defined with a nullable receiver type.

```
fun Any?.toString(): String {  
    if (this == null) return "null"  
    // after the null check, 'this' is autocast to a non-null type, so the toString() below  
    // resolves to the member function of the Any class  
    return toString()  
}
```

- Similarly to functions, Kotlin supports extension properties

val <T> List<T>.lastIndex: Int

get() = size - 1

Companion object extensions

```
class MyClass {  
    companion object { } // will be called "Companion"  
}  
  
fun MyClass.Companion.printCompanion() { println("companion") }  
  
fun main() {  
    MyClass.printCompanion()  
}
```

They can be called using only the class name as the qualifier:



Data classes |

- Classes whose main purpose is to hold data
`data class` User(**val** name: String, **val** age: Int)
- Compiler automatically derives the following members from all properties declared in the primary constructor:
 - ❑ `equals()/ hashCode()` pair
 - ❑ `toString()` of the form "User(name=John, age=42)"
 - ❑ `componentN()` functions corresponding to the properties in their order of declaration.
 - ❑ `copy()` function
- Have to fulfill the following requirements:
 - ❑ The primary constructor needs to have at least one parameter.
 - ❑ All primary constructor parameters need to be marked as **val** or **var**.
 - ❑ Data classes cannot be abstract, open, sealed or inner.

- *Component functions* generated for data classes enable their use in destructuring declarations

```
val jane = User("Jane", 35)
val (name, age) = jane
println("$name, $age years of age") // prints "Jane, 35 years of age"
```

Thank you

